

Linear systems (Chapter 3)

One goal: solve a system of n equations with n vari.

$$\begin{cases} 3x_1 + 7x_2 + x_3 = 2 \\ x_1 - x_2 = 0 \\ 4x_1 + 5x_2 + x_3 = 1 \end{cases} \quad \begin{matrix} 3 \times 3 \\ \text{system.} \end{matrix}$$

$$A \quad x = b$$

$$\begin{pmatrix} 3 & 7 & 1 \\ 1 & -1 & 0 \\ 4 & 5 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 2 \\ 0 \\ 1 \end{pmatrix}$$

Comes as a subroutine in other methods

Main problem:

$$Ax = b$$

$$A \in \mathbb{R}^{n \times n} \quad x \in \mathbb{R}^n \quad b \in \mathbb{R}^n$$

- ~~Cramer~~ (bad for implementation)
- Gaussian elimination ✓ stable, fast, sparse
- Computation of A^{-1} ✗ not stable, slower, sparse
- Iterative methods specific: sparse matrix

Numerical stability wrt roundoff errors.

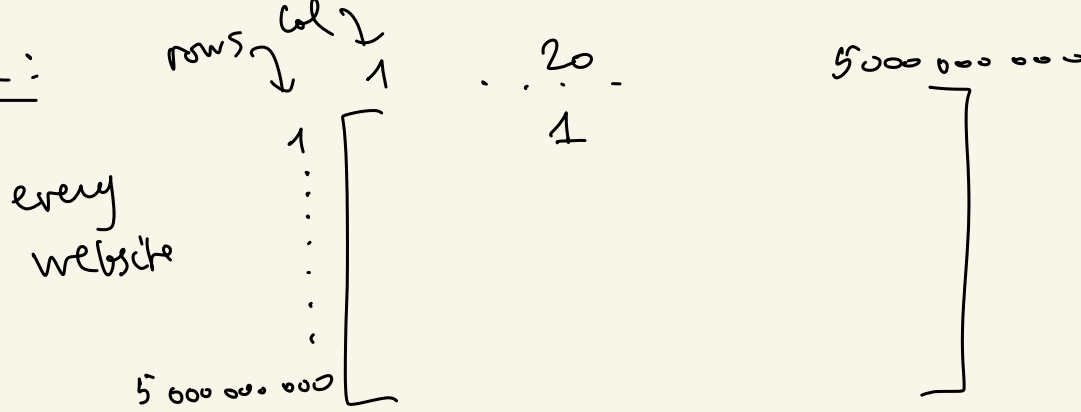
A matrix $A \in \mathbb{R}^{n \times n}$ is sparse when

A has a lot of 0.

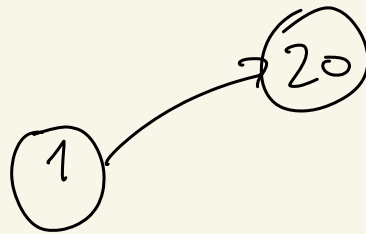
$n \approx 1000$
 1000000

In practice, A is very often sparse.

Example:



adjacency matrix of internet



on average
20 links in
each page

$10^{-6}\%$ of the elements are nonzero!

Sparsity $\approx \frac{20}{5\,000\,000\,000} \approx 10^{-8}$

How to store the web adjacency matrix

Sol 1: Store the value of each element

$$5 \cdot 10^9 \times 5 \cdot 10^9 = 25 \cdot 10^{18} \text{ elements to store}$$

1 byte / element : 25 000 000 TB (huge)

Sol 2: Store the places where the element is non zero and the value

20 . $5 \cdot 10^9$ non zero element

256 bytes / element 2 TB

Structure of the chapter

- Direct methods
Gaussian elimination, solution of triangular systems
- Analysis of the errors
Conditioning number, roundoff errors
- Iterative methods
Similar to the solution of nonlinear systems, useful for sparse matrices
- Regression (least-squares)
- Computation of the eigenvalues of a matrix

- 75% of the practical scientific problems make use of linear operators : these are **extremely frequent** problems
- Big difference between **theory** and **practice**
Theory : well known since a long time (the Greeks) but not always suitable to modern computers.
 Example : Cramer rules : complexity $n!$
Practice : only well known recently
 Good behaviour of the Gaussian elimination, LU decomposition.
- Obtaining a good large-scale linear software is a very difficult task

Direct methods

right-hand-side
rhs

Triangular system

Extremely simple to solve :

$$\begin{array}{l} 3x_2 + 2x_3 + x_4 = 2 \\ x_3 + 2x_4 = 3 \end{array} \quad \begin{pmatrix} 1 & 2 & 3 & 4 \\ 0 & 3 & 2 & 1 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & -1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix} \quad \checkmark \rightarrow \checkmark$$

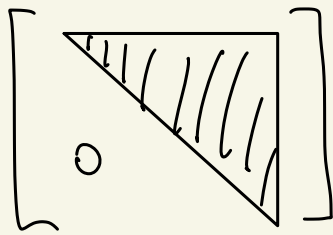
$$x_4 = -4$$

$$x_3 = 3 - 2x_4 \Rightarrow x_3 = 11$$

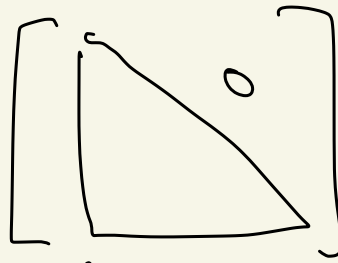
$$3x_2 = 2 - 2x_3 - x_4 \Rightarrow x_2 = -\frac{16}{3}$$

$$x_1 = 1 - 2x_2 - 3x_3 - 4x_4 \Rightarrow x_1 = -\frac{16}{3}$$

Backward substitution



upper triangular



lower

Triangular :

one row with 1 nonzero element: 47

one row with 2 nonzero el. including : 13, 47
the previous

:

one row — 3 — incl : 13, 47, 1

:

$$\begin{bmatrix} 1 & -1 & 0 \\ 0 & 2 & 0 \\ 1 & 2 & 2 \end{bmatrix}$$

is in a sense a triang. matr.

Computation of the complexity of a triangular solve:
 goal: count the number of operations.

1st step: 1 division

$$\text{2nd step: } x_{n-1} = \frac{b_{n-1} - a_{n-1,n} \times x_n}{a_{n-1,n-1}}$$

$$\text{3rd step: } x_{n-2} = \frac{b_{n-2} - a_{n-2,n-1} \times x_{n-1} - a_{n-2,n} \times x_n}{a_{n-2,n-2}}$$

⋮

$$\text{n}^{\text{th}} \text{ step: } x_1 = \frac{b_1 - \dots - a_{1,n} \times x_n}{a_{1,1}}$$

1

3

5

7

$2n-1$

In total: $1 + 3 + 5 + 7 + \dots + (2n-1) = n^2$

1	1
1+3	4
1+3+5	9
1+3+5+7	16
	25
	⋮
	⋮

$$\sum_{i=1}^n (2i-1) =$$

$$2 \underbrace{\sum_{i=1}^n i}_{2 \cdot \frac{n(n+1)}{2}} - \underbrace{\sum_{i=1}^n 1}_n$$

$$= n^2 + n - n = n^2$$

Solving a triangular system takes n^2 operations

How efficient?

Computer: 10^{10} /second

n	n^2	Time
$10^3 = 1000$	10^6	0
$10^4 = 10\,000$	10^8	0.01 sec
10^5	10^{10}	1 sec
$\leftarrow 10^6$	10^{12}	100 sec
$\leftarrow 10^7$	10^{14}	3 hours

does not
fit in
the RAM

A triangular system of size $10^6 \times 10^6$ is solvable
in 2 minutes.

Triangular systems

Computation of the number of operations in order to solve the triangular system :

$$\begin{array}{cccccccl} u_{11} x_1 & + & u_{12} x_2 & + & \cdots & + & u_{1n} x_n & = & b_1 \\ & & u_{22} x_2 & + & \cdots & + & u_{2n} x_n & = & b_2 \\ & & & & & & \vdots & & \\ & & & & & & u_{nn} x_n & = & b_n \end{array}$$

$$x_n = \frac{b_n}{u_{nn}}$$

$\vdots$

$$x_i = \frac{b_i - \sum_{k=i+1}^n u_{ik} x_k}{u_{ii}} \quad i = n, n-1, \dots, 1$$

$(n-i)$ additions , $(n-i)$ multiplications, 1 subtraction, 1 division

Total number of operations :

$$\begin{aligned} \sum_{i=1}^n (2n - 2i + 2) &= 2n^2 + 2n - 2 \sum_{i=1}^n i \\ &= 2n^2 + 2n - n(n+1) = n^2 + n \end{aligned}$$

$\mathcal{O}(n^2)$ operations

Gaussian elimination $\rightarrow$ triangularize the matrix

Principle : transform the rows of the matrix in order to obtain a triangular system that we then solve by backward substitution

Pivot $\rightarrow$ $\begin{pmatrix} 1 & 2 & 3 & 4 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 2 & 2 \\ 1 & -1 & 2 & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}$

1st step : We create 0's below the element $A(1,1)$

$$R_2 \leftarrow R_2 - R_1$$

$$R_4 \leftarrow R_4 - R_1$$

How do we triangularize?

Elementary row operations : valid operations that preserve the solution of your system

① Switch equations (rows of the matrix)

$$\begin{matrix} i \rightarrow \\ j \rightarrow \end{matrix} \begin{bmatrix} 1 & & & \\ & \ddots & & \\ & & 0 & 1 \\ & & 1 & 0 \\ & & & \ddots & 1 \end{bmatrix} A$$

② Multiply an equation by $m \neq 0$

$$\begin{bmatrix} 1 & & \\ & m & \\ & & \ddots \\ & & & 1 \end{bmatrix} A$$

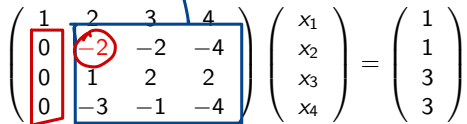
③ Replace $R_i \leftarrow R_i + m R_j$

$$\begin{matrix} i \rightarrow \\ j \rightarrow \end{matrix} \begin{bmatrix} 1 & & \\ & \ddots & \\ & & m \\ & & & 1 \end{bmatrix}$$

Gaussian elimination

start again

Principle : transform the rows of the matrix in order to obtain a triangular system that we then solve by backward substitution


$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 0 & -2 & -2 & -4 \\ 0 & 1 & 2 & 2 \\ 0 & -3 & -1 & -4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 3 \\ 3 \end{pmatrix}$$

2nd step : We create 0's below the element $A(2,2)$

$$R_3 \leftarrow R_3 + \frac{1}{2} R_2 \rightarrow -\frac{1}{(-2)}$$
$$R_4 \leftarrow R_4 - \frac{3}{2} R_2 \rightarrow -\frac{-3}{(-2)}$$

Gaussian elimination

Principle : transform the rows of the matrix **in order to obtain a triangular system** that we then solve by **backward substitution**

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 0 & -2 & -2 & -4 \\ 0 & 0 & \boxed{1} & 0 \\ 0 & 0 & 2 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ \frac{5}{2} \\ \frac{9}{2} \end{pmatrix}$$

3rd step : We create 0's below the element $A(3,3)$

$$R_4 \leftarrow R_4 - 2R_3$$

Gaussian elimination

After 3 steps: triangular system

Principle : transform the rows of the matrix in order to obtain a triangular system that we then solve by backward substitution

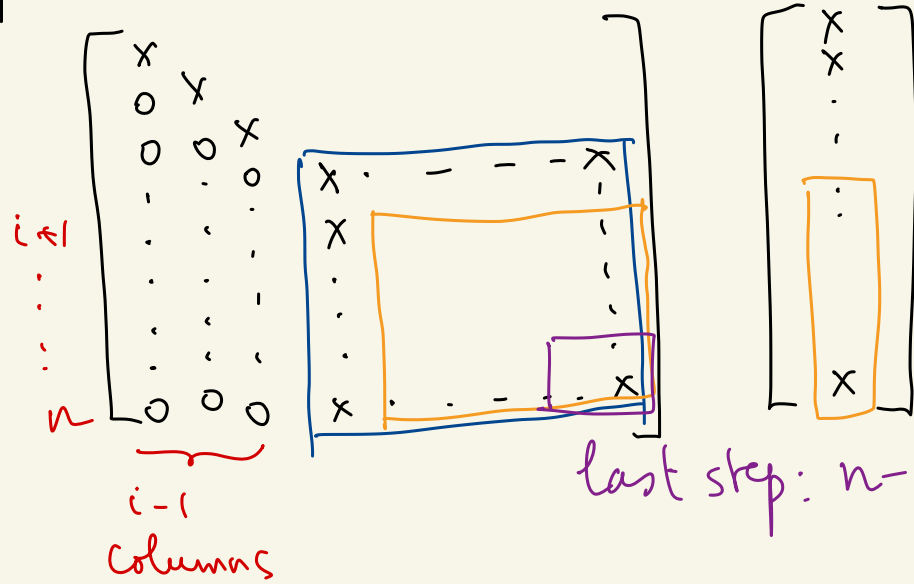
$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 0 & -2 & -2 & -4 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ \frac{5}{2} \\ -\frac{1}{2} \end{pmatrix}$$

We obtain a triangular system that we can solve by backward substitution

Solution :

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} -\frac{5}{2} \\ \frac{3}{2} \\ \frac{5}{2} \\ -\frac{1}{4} \end{pmatrix}$$

Counting the number of operations
Step i :



- ① Compute the multiplier: $m_{ji}^{(i)} = \frac{a_{ji}^{(i)}}{a_{ii}^{(i)}}$

1 div/row
($n-i$) rows
($n-i$) div.
- ② $R_j \leftarrow R_j - m_{ji}^{(i)} R_i$: 2 operations per element

($n-i$) rows and ($n-i+1$) columns

2($n-i$)($n-i+1$) operation

Summary:

Step i :

$$n-i + 2(n-i)(n-i+1) \text{ oper.}$$

$$= n-i + 2(n^2 - ni + n - ni + i^2 - i)$$

$$= 2i^2 + i(-4n-3) + (2n^2+3n)$$

Among all steps:

$$\sum_{i=1}^n$$

$$2i^2 + i(-4n-3) + (2n^2+3n)$$

$$\sum_{i=1}^n 2i^2 + \underbrace{\sum_{i=1}^n i(-4n-3)} + \underbrace{\sum_{i=1}^n (2n^2+3n)}$$

$$2 \frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)(-4n-3)}{2} + 2n^3 + 3n^2$$

What is the coefficient of the largest power?

$$2 \frac{2}{6} n^3 - 2n^3 + 2n^3 = \frac{2}{3} n^3 + \dots n^2 + \dots n + \dots$$

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$1$$

$$n=1: \frac{1 * 2 * 3}{6} = 1$$

$$1+4=5$$

$$n=2: \frac{2 * 3 * 5}{6} = 5$$

$$1+4+9=14$$

$$n=3: \frac{3 * 4 * 7}{6} = 14$$

$$1+4+9+16=30$$

⋮

Complexity of Gaussian elimination

$$\frac{2}{3} n^3$$

n	$\frac{2}{3} n^3$	Time (10^{10} op/sec)
1000	$\frac{2}{3} \cdot 10^9$	0.06 sec
10^4	$\frac{2}{3} \cdot 10^{12}$	66 sec
10^5	$\frac{2}{3} \cdot 10^{15}$	2 hours
10^6	$\frac{2}{3} \cdot 10^{18}$	60 000 000 sec . days...

only if it is
sparse we can
do better

Main problem: Solve a square linear system

$$Ax = b$$

$$A \in \mathbb{R}^{n \times n}$$

↓

(important: number of zeros)

n large

Gaussian elimination

① Triangularize

$\frac{2}{3} n^3$ operations
(assume A is full)
(no 0's)

② Backward substitution n^2 operations
(when the matrix is triangular)

Today

① Numerical stability



How robust we are towards numerical errors

(+ big topic next week)

② LU factorization

Complexity of the Gaussian elimination

Theorem

Consider the system $Ax = b$. If we apply the Gaussian elimination to obtain a triangular system, the required number of operations is

$$\frac{2}{3}n^3 + \frac{1}{2}n^2 - \frac{7}{6}n.$$

Proof : At step i , we compute, **for each row to eliminate**

1 division

$(n - i + 1)$ multiplications

$(n - i + 1)$ subtractions

There remains, at this stage, $(n - i)$ rows to eliminate

Gaussian elimination

① If an element that you want to be a pivot is 0 $\rightarrow$ need to switch rows

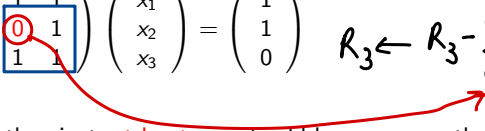
- The **pivot** is the element below which we create zeros!
- Problem** : if the pivot = 0

$$A^{(1)} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 1 & 2 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}$$

$R_2 \leftarrow R_2 - R_1$
 $R_3 \leftarrow R_3 - R_1$

$$\begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}$$

$R_3 \leftarrow R_3 - \frac{a_{32}^{(1)}}{a_{22}^{(1)}} R_2$



- Need to **switch rows**

Among all the elements below the pivot, **at least one** should be nonzero, otherwise the matrix is **singular**.

$$\begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$$

Gaussian elimination : choices of the pivots

- Risk of **numerical instability** if one of the pivots is **too close to 0**.
- Example :

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1.0001 & 2 \\ 1 & 2 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} \quad \begin{matrix} R_2 \leftarrow R_2 - R_1 \\ R_3 \leftarrow R_3 - R_1 \end{matrix}$$

- Without permutation of the rows, we obtain :

$$\begin{pmatrix} 1 & 1 & 1 \\ 0 & 0.0001 & 1 \\ 0 & 0 & 9999 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 10000 \end{pmatrix}$$

and a solution (with four significant digits)

$$(x_1, x_2, x_3) \approx (0, 0, 1)$$

instead of the true solution

$$(x_1, x_2, x_3) \approx (1, -1.0001, 1.0001).$$

- **Partial pivoting** : Choose the largest element of the current column *in abs. value*
Complete pivoting : Choose the largest element of the remaining submatrix *in abs. value*

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1.0001 & 2 \\ 1 & 2 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}$$

$$\begin{aligned} R_2 &\leftarrow R_2 - R_1 \\ R_3 &\leftarrow R_3 - R_1 \end{aligned}$$

$$\left. \begin{aligned} x_1 &= 1 - x_2 - x_3 \\ &\approx 0 \end{aligned} \right\} \begin{array}{l} \text{Solution} \\ x \approx \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \text{ BAD!} \end{array}$$

$$\begin{pmatrix} 1 & 1 & 1 \\ 0 & 0.0001 & 1 \\ 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}$$

$$R_3 \leftarrow R_3 - 10000 R_2$$

$$\begin{pmatrix} 1 & 1 & 1 \\ 0 & 0.0001 & 1 \\ 0 & 0 & -9999 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ -10000 \end{pmatrix}$$

$$\begin{aligned} -9999 x_3 &= -10000 \\ x_3 &= \frac{10000}{9999} \approx 1 \end{aligned}$$

4 sign. digits

$$x_3 \approx 1 \text{ (after roundoff error)}$$

$$0.0001 x_2 = 1 - x_3 \approx 0$$

$$\Rightarrow x_2 = 0 \text{ (previous roundoff error is multiplied by 10000)}$$

$$\begin{pmatrix} 1 & 1 & 1 \\ 0 & 0.0001 & 1 \\ 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}$$

Avoid the small pivot
Switch row 2 and 3.

$$\begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0.0001 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$$

$$R_3 \leftarrow R_3 - 0.0001 R_2$$

$$0.9999 x_3 = 1$$

$$x_3 = 1.0001$$

$$x_3 \approx \frac{1}{0.9999} \approx 1$$

$$\begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0.9999 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \rightarrow$$

$$x_2 = -x_3 \approx -1$$

$$x_2 = -1.0001$$

$$x_1 = 1 - x_2 - x_3 \approx 1$$

$$x = \begin{pmatrix} 1 \\ -1 \\ 1 \end{pmatrix}$$

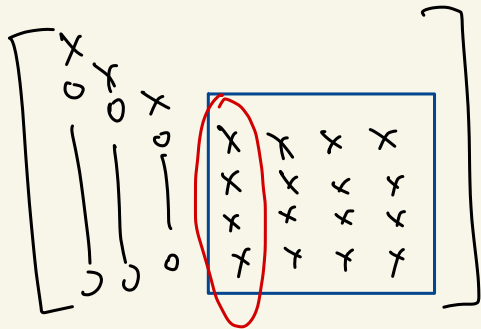
$$\text{Accurate: } x = \begin{pmatrix} 1 \\ -1.0001 \\ 1.0001 \end{pmatrix}$$

$$x = 1$$

Conclusion: We should always avoid too small pivots

How?

① Partial pivoting

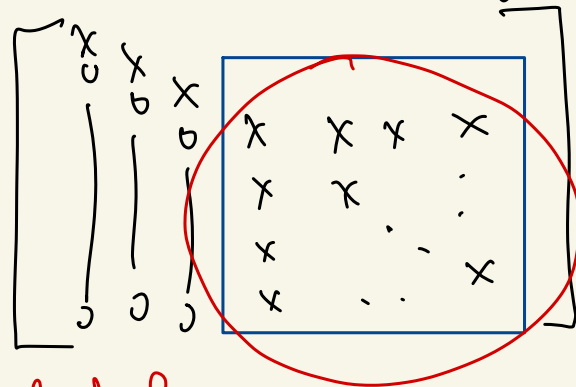


look for
the most stable
pivot in the column
(= the largest in
abs. value)

$$\begin{aligned} & n-1 \\ & + n-2 \\ & + n-3 \\ & + \dots \\ & + 1 \\ & = \frac{n(n-1)}{2} \end{aligned}$$

$$\sim \frac{n^2}{2} \downarrow \text{cheap}$$

② Complete pivoting



look for
the most stable
pivot in the whole
remaining matrix.

$$\begin{aligned} & (n-1)^2 + (n-2)^2 + \dots + 1 \\ & = \frac{(n-1)n(2n-1)}{6} = \frac{1}{3}n^3 \end{aligned}$$

$$\frac{2}{3}n^3 + \frac{1}{3}n^3 \rightarrow n^3 \quad (\text{adds } \frac{1}{3} \text{ of time})$$

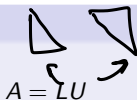
How expensive?

LU decomposition

Gaussian elim. : ① Triangul. $\frac{2}{3}n^3$
② Substit. n^2

Principle

We **factorize** the matrix A in


$$A = LU$$

where L is **lower triangular** and U is **upper triangular**. We solve two **triangular** systems

$$Ly = b \quad \text{and} \quad Ux = y.$$

- **Complexity** : $2n^2$
- Gaussian elimination : $\frac{2}{3}n^3 + \frac{1}{2}n^2 - \frac{7}{6}n$

$Ax = b$ and you know $A = LU$

$$L \underbrace{Ux}_{y \in \mathbb{R}^n \text{ unknown}} = b$$

$$Ly = b$$

Solve it in n^2 oper.
and find y

$$Ux = y$$

Solve it in n^2 oper
and find x .

Find x in $2n^2$ operations.

LU factorization is obtained with Gaussian elimination.

$$A^{(0)} = \begin{bmatrix} x & x & \dots & x \\ \vdots & & & \vdots \\ x & \dots & \dots & x \end{bmatrix}$$

$$A^{(1)} = \begin{bmatrix} x & x & \dots & x \\ 0 & x & \dots & x \\ \vdots & \vdots & & \vdots \\ 0 & x & \dots & x \end{bmatrix} =$$

$$A^{(2)} = \begin{bmatrix} x & x & \dots & x \\ 0 & x & x & \dots & x \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 0 & x & \dots & x \end{bmatrix} =$$

...

$$R_2 \leftarrow R_2 + m_{21} R_1$$

$$R_n \leftarrow R_n + m_{n1} R_1$$

$$\begin{bmatrix} 1 & & & \\ m_{21}^{(1)} & & & \\ \vdots & & & \\ m_{n1}^{(1)} & & & 1 \end{bmatrix}$$

$A^{(0)}$

(translation in matrix form)

$$\begin{bmatrix} 1 & & & \\ & 1 & & \\ m_{32}^{(2)} & & & \\ \vdots & & & \\ m_{n2}^{(2)} & & & 1 \end{bmatrix} A^{(1)}$$

$$\begin{bmatrix} 1 & & & \\ & \ddots & & \\ & & \ddots & \\ & & & 1 \\ & & m_{n,n-1} & 1 \end{bmatrix} \cdots \begin{bmatrix} 1 & & & \\ & 1 & & \\ & m_{32} & \ddots & \\ & \vdots & & \ddots & \\ & m_{n2} & & & 1 \end{bmatrix} \begin{bmatrix} 1 & & & \\ & m_{21} & \ddots & \\ & \vdots & & \ddots & \\ & m_{n1} & & & 1 \end{bmatrix} A = \underbrace{\begin{bmatrix} x & \ddots & \ddots & x \\ & \ddots & & \vdots \\ & & \ddots & \vdots \\ & & & x \end{bmatrix}}_U$$

$E^{(n-1)} \quad \dots \quad E^{(2)} \quad E^{(1)}$

$$(E^{(n-1)})^{-1} = \begin{bmatrix} 1 & & & \\ & \ddots & & \\ & & \ddots & \\ & & & 1 \\ & m_{n,n-1} & & 1 \end{bmatrix} \cdots (E^{(1)})^{-1} = \begin{bmatrix} 1 & & & \\ -m_{21} & \ddots & & \\ \vdots & & \ddots & \\ -m_{n1} & & & 1 \end{bmatrix}$$

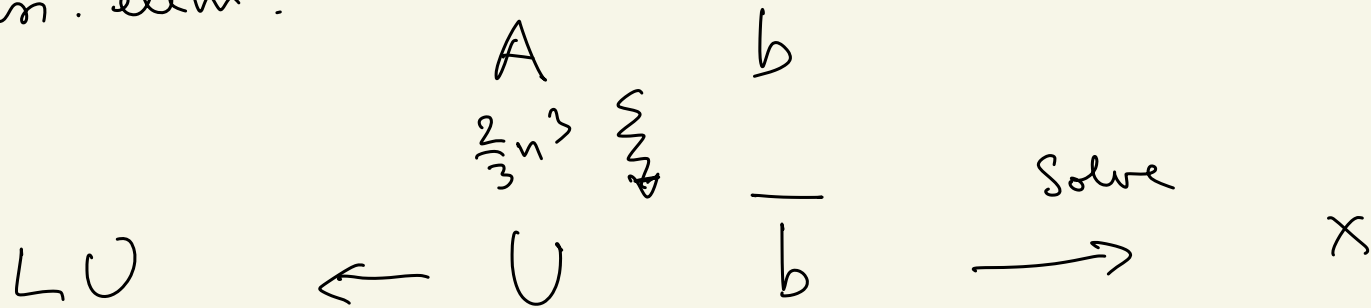
$$A = (E^{(1)})^{-1} (E^{(2)})^{-1} \cdots (E^{(n-1)})^{-1} U$$

$$A = \begin{bmatrix} 1 & & & \\ -m_{21} & \ddots & & \\ \vdots & \ddots & \ddots & \\ -m_{n1} & \dots & -m_{n,n-1} & 1 \end{bmatrix} \begin{bmatrix} & & & \\ & \cup & & \\ & & & \\ & & & \end{bmatrix}$$

Gaussian elimination directly provides the LU factorization.

L is obtained as the identity with $\rightarrow$ multipliers below the diagonal

Gauss. elim.



$$A x = c \quad (\text{same } A)$$

$$L y = c \quad \rightarrow \quad y = \bar{c} \quad (\text{redoing the Gaussian elim on the rhs})$$

$$U x = \bar{c} \quad \rightarrow \quad \text{solve}$$

LU factor. is simply a way to store the operations of Gaussian elim.

Very useful if we need to solve

Several systems with the same A different rhs.

once full process $\frac{2}{3}n^3 \rightarrow L$ and U

each time we have a new rhs $\rightarrow$ reuse L and $U \rightarrow 2n^2$

① Triangularization $\frac{2}{3}n^3$

→ L and U

② $Ly = b$

n^2

→ y

③ $Ux = y$

n^2

→ x

$$\frac{2}{3}n^3 + 2n^2$$

LU decomposition and Gaussian elimination

Theorem

Let A be a square matrix of size n for which the Gaussian elimination can be performed without any permutation of the rows. Then the matrix has a LU decomposition such that the elements of L and U are given by the elements of the Gaussian elimination.

In practice

$$L = (m_{ik}), \quad i \geq k, \quad U = (a_{kj}^{(k)}), \quad k \leq j$$
$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \rightarrow \begin{pmatrix} a_{11}^{(n)} & a_{12}^{(n)} & \cdots & a_{1,n-1}^{(n)} & a_{1n}^{(n)} \\ m_{21} & a_{22}^{(n)} & \cdots & a_{2,n-1}^{(n)} & a_{2n}^{(n)} \\ \vdots & \vdots & & \vdots & \vdots \\ m_{n1} & m_{n2} & \cdots & m_{n,n-1} & a_{nn}^{(n)} \end{pmatrix}$$

Inverse matrix

A

Compute A^{-1}

New rhs b , compute $A^{-1}b$
which gives the solution

Next: Compare the use of LU factoriz.
 A^{-1}

to solve a system

LU vs A^{-1}

- ① Time needed to obtain the tool
Space
- ② Time needed to solve $Ax=b$ with the tool
- ③ What if A is sparse (lots of 0's)
- (④ Numerical stability when obtaining the tool)

LU vs Matrix Inverse

$$\frac{2}{3}n^3 \quad \text{vs} \quad n^3$$

Initial computation

$LU \rightarrow$ (Gauss elimination) $\frac{2}{3}n^3$ operations

Inverse $\rightarrow$ either LU with n solves $\frac{2}{3}n^3 + nn^2 = \frac{5}{3}n^3$
or Gauss-Jordan form n^3 operations

- Similar complexity but LU is cheaper.
- Memory space is the same : n^2

Winner: LU

A . How to obtain A^{-1}

$$* A^{-1} = \frac{1}{\det(A)} \begin{bmatrix} \text{principal minors} \\ \text{of } A \\ \text{(cofactors)} \end{bmatrix}^T$$

1 determinant n^2 computations of a determinant

(n^2+1) determinants to compute

$\rightarrow$ Time?

Bad: apply the def. $n!$ coeff.

Good: triangularizing the matr. n^3

$$* A^{-1} = \left[A \mid I \right] \xrightarrow{\text{triang. Gauss-Jordan form}} \left[I \mid A^{-1} \right] \rightarrow n^3$$

LU vs Matrix Inverse

LU solve for a dense matrix and dense right-hand-side

- $Ly = b$
Because of the 1 on the diagonal, $n^2 - n$ operations.
- $Ux = y$
Plain triangular solve, n^2 operations.
- Total : $2n^2 - n$ operations.

Use of inverse for a dense matrix and dense right-hand-side

- Matrix-vector multiplication
- n elements to compute. Each element needs n multiplications and $n - 1$ additions.
- Total : $2n^2 - n$ operations.

$$A^{-1} * b$$

2n - n per elem.

Criterion 2 : Draw ✓

LU vs Matrix Inverse (the sparse case)

Assume A has a lot of 0's.

Tri-diagonal matrices

$3n$ nonzero elements instead of n^2

$$R_2 \leftarrow R_2 - \frac{a_{21}}{a_{11}} R_1$$

$$A = \begin{pmatrix} a_{11} & a_{12} & & & 0 \\ a_{21} & a_{22} & a_{23} & & \\ & a_{32} & a_{33} & a_{34} & \\ 0 & & \ddots & \ddots & \ddots \\ & & & a_{n-1,n-1} & a_{n-1,n} \\ & & & a_{n,n-1} & a_{nn} \end{pmatrix}$$

Sparsity of $\frac{3n}{n^2} = \frac{3}{n}$.

Special class of **very sparse** matrices

The L and U are **bidiagonal**.

$$A = \begin{pmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ 0 & l_{32} & 1 & & \\ & & \ddots & \ddots & \\ & & & l_{n,n-1} & 1 \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} & & & \\ & u_{22} & u_{23} & & \\ & & u_{33} & u_{34} & \\ & & & \ddots & \ddots \\ & & & & u_{n-1,n-1} & u_{n-1,n} \\ & & & & & u_{nn} \end{pmatrix}$$

Substitution :

$$Ly = b$$

2 operations per row

$2n$ operations

$$Ux = y$$

$O(n)$ operations

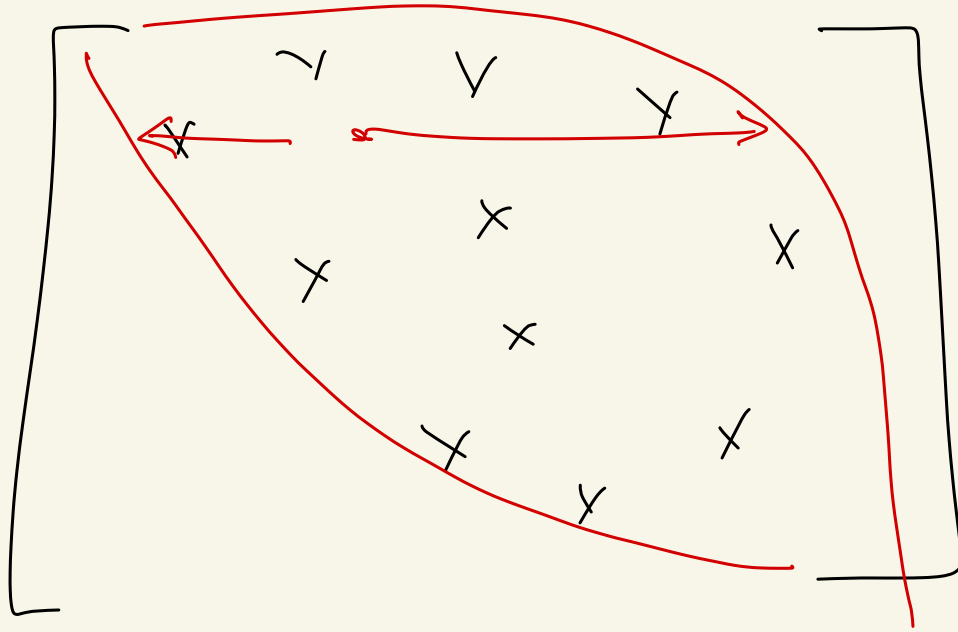
Obtaining L and U : $O(n)$
Substituting $O(n)$

Inverse matrix is dense in general.

- L and U need $3n$ of memory-space.
Solving the triangular systems takes only $\mathcal{O}(n)$ operations.
- The inverse matrix is in general **fully dense**! (or close to fully dense)
Multiplying by the inverse requires $\mathcal{O}(n^2)$ operations.
 ↳ The elements may be less numerically stable
- Generalizes to arbitrary sparse matrices.
 LU is often (but not always) more compact.
- Computation of the inverse may be **numerically unstable**
- LU is **more flexible** because one can play with **permutations** or blocks that are **not decomposed**.

↓
decrease
the bandwidth

When is LU efficient?



bandwidth
(distance
around
the diagonal
when there
are nonzeros)

A

You can possibly switch rows to decrease the bandwidth
(hard problem)

Never compute the inverse of a matrix !

N.B. If you do compute an inverse in this lecture, you will be penalized !

You can **write** $A^{-1}b$ to **mean** solve the system $Ax = b$.

Use **LU factorization** to solve a system !

The diagram consists of three curved arrows originating from the 'LU' part of 'LU factorization' and pointing towards the handwritten text 'Gauss. elim.', 'LU', and 'iterative meth.' respectively.

Solving

$$Ax = b$$

$$A \in \mathbb{R}^{n \times n}$$

① Gaussian elimination

1. Triangularization $\frac{2}{3}n^3$

2. Backward subst. n^2

($A = LU$
as byproduct)

② Clever pivoting in order to avoid numerical issues

③ A^{-1} is to be avoided numerically

Goal today: $Ax = b$

Discussion: errors in the computation

$$Ax = b$$



$$(A + \delta A)x = b$$

small variation in A



$$Ax = b + \delta b$$

small variation in b

(on this slide, we assume that we can solve with no roundoff error)

Sensitivity to data errors

Some **linear problems** are **ill-conditioned**.

A small perturbation of the **data** leads to a large variation of the solution.

Example :

Consider the system

$$\begin{pmatrix} 51 & 30 & 50 \\ 102 & 59 & 100 \\ 152 & 90 & 149 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 4 \end{pmatrix}.$$

Solution : $x = (-9 \ 2 \ 8)^T$.

Small error in the matrix:

$$\begin{pmatrix} 50.99 & 30 & 50 \\ 102 & 59 & 100 \\ 152 & 90 & 149 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 4 \end{pmatrix}.$$

New solution $x = (-2.91 \ 1.94 \ 1.83)^T$

Small
perturbation



large
variation
in the sol.

Some systems do not behave like that

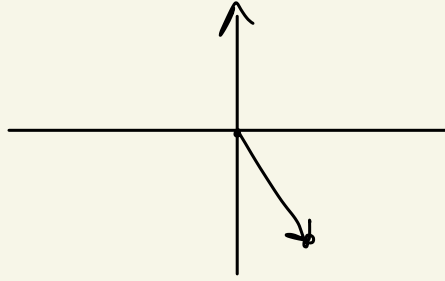
$$\begin{pmatrix} 1 & & \\ & 2 & \\ & & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \quad \text{Sol} = \begin{pmatrix} 1 \\ 1/2 \\ 1/3 \end{pmatrix}$$

$$\begin{pmatrix} 0.99 & & \\ & 2 & \\ & & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \quad \text{Sol} \approx \begin{pmatrix} 1.01 \\ 1/2 \\ 1/3 \end{pmatrix}$$

- Task:
- ① Have a way to detect ill-conditioned systems
 - ② Make the variation more precise

Norm indicates a length

$$v = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$



$$\|v\| = \sqrt{1^2 + (-2)^2} = \sqrt{5}$$

Matrix norms

Definition

(math. def.)

$$\|\cdot\|: \mathbb{R}^n \rightarrow \mathbb{R}$$

We say that $\|x\|$ is a **vector norm** if the following axioms are satisfied.

- $\|x\| > 0$ for all $x \neq 0$ and $\|x\| = 0$ implies $x = 0$
- $\|x + y\| \leq \|x\| + \|y\|$ (triangular ineq.)
- $\|\alpha x\| = |\alpha| \|x\|$ for all $\alpha \in \mathbb{R}$

Definition

$$\|\cdot\|: \mathbb{R}^{n \times n} \rightarrow \mathbb{R}$$

We say that $\|A\|$ is a **matrix norm** if the following axioms are satisfied.

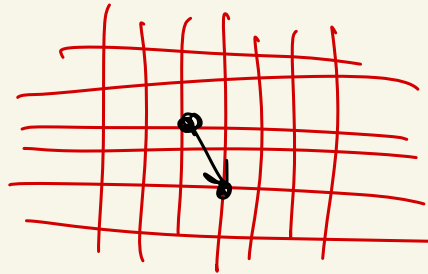
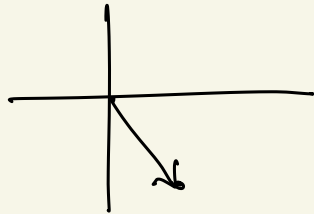
- $\|A\| > 0$ for all $A \neq 0$, and $\|A\| = 0$ implies $A = 0$
- $\|A + B\| \leq \|A\| + \|B\|$
- $\|\alpha A\| = |\alpha| \|A\|$ for all $\alpha \in \mathbb{R}$

If furthermore, the following two axioms are satisfied :

- $\|Ax\| \leq \|A\| \|x\|$
- $\|AB\| \leq \|A\| \|B\|$

then we say that the matrix norm $\|A\|$ is **compatible** with the vector norm $\|x\|$.

$$x = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$



$$\|x\|_2 = \sqrt{1^2 + (-2)^2} = \sqrt{5}$$

$$\|x\|_1 = |1| + |-2| = 3$$

$$\|x\|_\infty = \max \{ |1|, |-2| \} = 2$$

$$l_p\text{-norm} = \left(\sum_i |x_i|^p \right)^{1/p}$$

Make a connection between vector-norm
matrix-norm

Assume $\|\cdot\|_*$ is a vector norm

$$\|A\|_* = \max_{v: \underbrace{\|v\|_*}_{\text{vector}} = 1} \underbrace{\|Av\|_*}_{\text{vector}}$$

then $\|A\|_*$ is compatible with the vector norm

Matrix norms : examples

4 most usual matrix norms

- $\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}|$ *max of 1-norm of the columns*
- $\|A\|_2 = (\text{largest eigenvalue of } A^T A)^{1/2}$ *largest singular value of A*
- $\|A\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}|$ *max of 1-norm of the rows*
- $\|A\|_F = \left(\sum_{i,j=1}^n |a_{ij}|^2 \right)^{1/2}$

Frobenius

Example

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

$$\|A\|_1 = \max\{1 + 4 + 7, 2 + 5 + 8, 3 + 6 + 9\} = 18,$$

$$\begin{aligned} \|A\|_2 &= \max\{\text{eigenvalue of } A^T A\}^{1/2} \\ &= \max\{0, 1.07, 16.85\} \approx 16.85 \end{aligned}$$

$$\|A\|_\infty = \max\{1 + 2 + 3, 4 + 5 + 6, 7 + 8 + 9\} = 24$$

$$\|A\|_F = \sqrt{1 + 4 + 9 + 16 + \dots + 81} \approx 16.88.$$

Perturbation of the data

Condition number $\rightarrow$ indicates whether good or ill-conditioned

$$\kappa(A) = \|A\| \|A^{-1}\|$$

Proposition

Let $A \in \mathbb{R}^{n \times n}$ be a nonsingular matrix, $b \in \mathbb{R}^n$ and let $\bar{x} \in \mathbb{R}^n$ be the solution of $Ax = b$. We can bound the relative error on x when solving $Ax = (b + \delta b)$ instead of $Ax = b$, by

New sol:
 $\bar{x} + \delta \bar{x}$

$$\frac{\|\delta x\|}{\|x\|} \leq \kappa(A) \frac{\|\delta b\|}{\|b\|} \sim \text{relative error on } b.$$

relative error on x

Proposition

Let $A \in \mathbb{R}^{n \times n}$ be nonsingular matrix, let $b \in \mathbb{R}^n$ and let $x \in \mathbb{R}^n$ be the solution of $Ax = b$. We can bound the relative error on x when solving $(A + \delta A)x = b$ instead of $Ax = b$, by

$$\frac{\|\delta x\|}{\|x + \delta x\|} \leq \kappa(A) \frac{\|\delta A\|}{\|A\|}.$$

$$\kappa(A) = \|A\| \|A^{-1}\|$$

In the case of a 2-norm, we can compute it without knowing A^{-1} .

Properties: $\kappa(A) \geq 1$, the smaller the better

$$A = \begin{pmatrix} 1 & & \\ & 1 & \\ & & \ddots \\ & & & 1 \end{pmatrix}$$

$$\|A\|_1 = 1$$

$$A^{-1} = \begin{pmatrix} 1 & & \\ & \ddots & \\ & & 1 \end{pmatrix}$$

$$\|A^{-1}\|_1 = 1$$

$$\kappa \begin{pmatrix} 1 & & \\ & \ddots & \\ & & 1 \end{pmatrix} = 1$$

Large condition number make things bad.

$$A \delta \bar{x} = \delta b$$

$$\delta \bar{x} = A^{-1} \delta b$$

$$\|\delta \bar{x}\| \leq \|A^{-1}\| \|\delta b\| \quad (1)$$

all members of the ineq. > 0 ,
I can multiply them

$$A \bar{x} = b$$

$$\|b\| \leq \|A\| \cdot \|\bar{x}\|$$

$$\text{Since } \|b\| > 0 \\ \|\bar{x}\| > 0$$

$$\frac{1}{\|\bar{x}\|} \leq \frac{\|A\|}{\|b\|} \quad (2)$$

$$\frac{\|\delta \bar{x}\|}{\|\bar{x}\|} \leq \|A^{-1}\| \cdot \|A\| \cdot \frac{\|\delta b\|}{\|b\|}$$

Example of an ill-conditioned matrix

$$\begin{pmatrix} 53.99 & \cdot & \cdot \\ \vdots & \ddots & \vdots \end{pmatrix}$$

$$\delta A = \begin{pmatrix} 0.01 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

$$A = \begin{pmatrix} 51 & 30 & 50 \\ 102 & 59 & 100 \\ 152 & 90 & 149 \end{pmatrix}$$

$$\|A\|_{\infty} = 391$$

$$A^{-1} = \begin{pmatrix} -209 & 30 & 50 \\ 2 & -1 & 0 \\ 212 & -30 & -51 \end{pmatrix}$$

$$\|A^{-1}\|_{\infty} = 293$$

$$\kappa(A) = 391 \times 293 = 114563 \approx 10^5$$

$$A^{-1} = \frac{1}{\det(A)} \begin{bmatrix} & & \\ & & \\ & & \end{bmatrix}^T$$

$$\det(A) = 1$$

$$\frac{\|\delta A\|}{\|A\|} = \frac{0.01}{391} \approx 2.5 \cdot 10^{-5}$$

Hence

$$\frac{\|\delta x\|}{\|x + \delta x\|} \leq 2.8 = \kappa(A) \cdot \frac{\|\delta A\|}{\|A\|}$$

We had $x + \delta x = (-2.9 \ 1.9 \ 1.8)^T$ and $\delta x \approx (6.1 \ -0.1 \ -6.2)^T$ and

$$\frac{\|\delta x\|}{\|x + \delta x\|} \approx \frac{6.2}{2.9} \approx 2.1$$

210% error
in the difference
 $\bar{x}$ and $\bar{x} + \delta \bar{x}$

$\kappa(A)$ is large $\rightarrow$ small perturbation of the data
 (or a roundoff error)
 $\Downarrow$
 big perturbation of the solution

$$Ax = b \xrightarrow{\text{solution}} \bar{x}$$

Question: is $\bar{x}$ accurate?

$$\underbrace{A\bar{x} - b}_{\delta b} = \text{residual} = r = \begin{pmatrix} 10^{-7} \\ 10^{-9} \\ 10^{-20} \\ \vdots \end{pmatrix}$$

$\delta b \rightarrow$ Conclusion: we have solved $Ax = b + r$
 $\frac{\| \delta x \|}{\| x \|} \leq \frac{\| r \|}{\| b \|} \cdot \kappa(A)$

Roundoff errors in the Gaussian elimination

Proposition

Let $\bar{L} = (\bar{m}_{ik})$ and $\bar{U} = (\bar{a}_{kj}^{(n)})$ be the triangular matrices obtained by the Gaussian elimination algorithm. There exists a matrix E such that $\bar{L}, \bar{U}$ are the exact decomposition of $A + E$, i.e.

$$\bar{L} \bar{U} = A + E$$

If we use a (partial or complete) pivoting strategy and a floating point arithmetic with a machine epsilon ϵ_M then the matrix E is bounded by

$$\|E\|_{\infty} \leq n^2 \underbrace{g_n}_{\epsilon_M} \|A\|_{\infty}$$

where

$$g_n = \frac{\max_{i,j,k} |\bar{a}_{ij}^{(k)}|}{\max_{i,j} |a_{ij}|}.$$

good strategy
is to maintain
small numbers
during Gaus-
elim.

0.01

1 2

$R_{17} \sim 100 R_{16}$

Roundoff errors in the solution of a triangular system

Proposition

Consider the linear system $Lx = b$ where $L = (l_{ij})$ is a lower triangular matrix. The vector $\bar{x}$ computed by a forward substitution is the exact solution of the perturbed triangular system

$$(L + \delta L)\bar{x} = b$$

where the perturbation matrix δL is bounded by

$$\|\delta L\|_{\infty} \leq \frac{n(n+1)}{2} \epsilon_M \max_{i,j} |l_{ij}|$$

Scaling of the equations : good practice to avoid numerical problems

- Scaling of a variable : $x_j = \alpha_j x'_j$
- Scaling of a right-hand-side : multiplication of the j^{th} equation by β_j
- New system : $A'x' = b'$ where $x'_i = \frac{x_i}{\alpha_i}$ i.e. $D_2 A D_1 x' = D_2 b$

$$\begin{pmatrix} \beta_1 & & \\ & \ddots & \\ & & \beta_n \end{pmatrix} A \begin{pmatrix} \alpha_1 & & \\ & \ddots & \\ & & \alpha_n \end{pmatrix} \begin{pmatrix} x'_1 \\ \vdots \\ x'_n \end{pmatrix} = \begin{pmatrix} \frac{b_1}{\beta_1} \\ \vdots \\ \frac{b_n}{\beta_n} \end{pmatrix}$$

Theorem

Let $\bar{x}$ and $\bar{x}'$ be the two solutions of $Ax = b$ and $(D_2 A D_1)x' = D_2 b$. If D_1 and D_2 are diagonal matrices whose elements are **integer powers** of the numeral system (**often 2**) (hence no roundoff errors from the scaling) then the Gaussian elimination performed in floating point arithmetic produce exactly the same solution up to the used scaling, assuming that we use the **same pivots**.

Not true if the scaling induces different choices of pivots !

Badly scaled: Coefficients have different order of magnitude

$$x_1 - 0.0001 x_2 + 93252 x_3 = 172$$

Coeff.

$$500000 x_1 - 499987 x_2 = 1$$

↑ ↑
2 large values that are subtracted

Useful: change the scale of a variable

$$\bar{x}_i := \frac{x_i}{10^6}$$

Change the scale of an equation: multiply a row by a value

$$A = \begin{pmatrix} 1000000 & & \\ & 1 & \\ & & 1 \end{pmatrix}$$

$$A^{-1} = \begin{pmatrix} \frac{1}{1000000} & & \\ & 1 & \\ & & 1 \end{pmatrix}$$

$$\|A\| = 1000000$$

$$\|A^{-1}\| = 1$$

$$K(A) = 1000000$$

Scaling

$$\bar{A} = \begin{pmatrix} 1 & & \\ & 1 & \\ & & 1 \end{pmatrix}$$

$$K(\bar{A}) = 1$$

Woodclap quiz

①

$$A^{-1}b : 2n^2 - n \text{ op.}$$

$$Ly = b : n^2 - n \quad \left\{ \begin{array}{l} 2n^2 - n \text{ op.} \end{array} \right.$$

$$Ux = y : n^1$$

Same for
dense

But for sparse, we can expect L, U to be more sparse.

②

The formula tells you everything.

You have to look at $K(A) \cdot \frac{\|r\|}{\|b\|}$.

③ Gaussian elimination: $\frac{2}{3}n^3$

Complete pivoting: $\frac{1}{3}n^3$

In total: n^3 : only the coefficient increases.

④ Dense: Triang: n^3
Subst: n^2 so indeed negligible

General Sparse : True as well

Banded : same time

⑤ $\kappa(A)$ indicates the impact of roundoff errors.

⑥ $\left(\overline{\quad} \right)$
0.01

Keeping a small pivot will then multiply roundoff errors by a large value

2 things:

① Condition number plays a role even for exact comput.

② Even for small condition number, a small pivot may increase the roundoff errors.

Dealing with sparse matrices in practice (details of implementation)

Sparse matrix

$$A = \begin{bmatrix} 2.7 & x & x \\ & x & \\ & & 1.9 \\ & x & \end{bmatrix}$$

① Important to only store nonzero values

Idea 1: in random order

$$\begin{bmatrix} (1,3): 2.7 \\ (19,22): 1.9 \\ \vdots \end{bmatrix}$$

Av is hard to do if you have random order.

Idea 2: Stored in compressed column - format

The matrix is stored column by column

$$A = \begin{bmatrix} 1.9 & & 3.1 \\ 2.3 & & \\ 2.4 & & \\ & & \end{bmatrix}$$

Access to a column: FAST
Access to a row: "SLOW"

rows = [1, 3, 4, 1]

columns = [1, 2, 2, 3]

values = [1.9, 2.3, 2.4, 3.1]

where in the "row" vector
does each column start
(pointer to the start of
a column)

col ptr: [1, 2, 4]

col. 1 starts at 1
col. 2 starts at 2
col. 3 starts at 4

With a sparse matrix:

$A v$ is efficient
(multiplication of a matrix and a vector)

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

$$= \begin{bmatrix} (1 \ 2 \ 3) \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} = 14 \\ (4 \ 5 \ 6) \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} = 32 \\ \vdots \end{bmatrix}$$

$$= \begin{bmatrix} 1 \\ 4 \\ 7 \end{bmatrix} * 1 + \begin{bmatrix} 2 \\ 5 \\ 8 \end{bmatrix} * 2 + \begin{bmatrix} 3 \\ 6 \\ 9 \end{bmatrix} * 3$$

It seems that we need
an access to a row!

Matrix vector written
in such a way that
we only access the column.

With a sparse matrix

efficient operation : Matrix Vector mult.

Next week: Solve $Ax=b$
by only doing matrix-vector.
mult.

$$\begin{cases} x_1 + 4x_2 + 7x_3 = 1 \\ 2x_1 + 5x_2 + 8x_3 = 2 \\ 3x_1 + 6x_2 + 9x_3 = 3 \end{cases}$$

$$Ax = b$$

$$A = \begin{pmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{pmatrix} \quad b = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

$$\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} x_1 + \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix} x_2 + \begin{pmatrix} 7 \\ 8 \\ 9 \end{pmatrix} x_3 = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

access to
the columns
are
highlighted

Computation with sparse matrix

$$\begin{bmatrix} x & & & & \\ & x & & & \\ & & x & & \\ & & & x & \\ & x & & & \\ & & x & & \\ & & & x & \end{bmatrix}$$

$$A \in \mathbb{R}^{n \times n}$$

n large

A has lots of zeros

Only store the nonzeros
(column by column)

Goal of today:

Solve $Ax = b$
without changing A

Operation to prioritize:

$$Av$$

(cheap for a
sparse matrix)

Iterative methods

- In some cases, we have **huge** linear systems that are very **sparse**.

- The Gaussian elimination is not very appropriate because of the **fill in** :

Initial matrix :

Large bandwidth

$$\begin{pmatrix} \textcircled{\times} & \times & \times & \cdots & \times \\ \times & \times & & \cdots & \\ \times & & \times & \cdots & \\ \vdots & & & \ddots & \\ \times & & & \cdots & \textcircled{\times} \end{pmatrix}$$

Fill in rate : $\frac{3n}{n^2}$

After one iteration of the Gaussian elimination :

$$\begin{pmatrix} \times & \times & \times & \cdots & \times \\ \times & \times & \times & \cdots & \times \\ \times & \times & \times & \cdots & \times \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \times & \times & \times & \cdots & \times \end{pmatrix}$$

Fill in rate : potentially 100 %

- We would like to keep the **sparse property** and perform **cheap operations**, e.g. matrix-vector multiplication.

- → iterative methods.

Problem : does not always converge !

Fixed-point iteration

Look for a fixed-point of a function

f

$$x = f(x)$$

Init: x_0

Iteration: $x_{k+1} = f(x_k)$

$$\lim_{k \rightarrow \infty} x_k = \bar{x} \quad \text{with} \quad \bar{x} = f(\bar{x})$$

$$x = \cos(x)$$

$$x_{k+1} = \cos(x_k)$$

$$F: \mathbb{R}^n \rightarrow \mathbb{R}^n$$

Goal: transform

$$Ax = b$$

Rewrite $x = G(x)$

Iterative methods : principle

$$Ax = b$$

$$0 = b - Ax$$



Idea : based on a fixed-point method

We write $Ax = b$ as

$$Qx = Qx - Ax + b$$

For a **clever choice** of Q , the system (1), **Solve the system in Q .** (1)

$$x^{(k+1)} = Q^{-1}((Q - A)x^{[k]} + b)$$

- is **quick** to solve **at each iteration**
- $x^{(k)}$ converges to the **solution** of $Ax = b$

Choose Q to be diagonal $\rightarrow$ Jacobi
Choose Q to be triangular $\rightarrow$ Gauss-Seidel

$$Qx^{(k+1)} = \underbrace{(Q-A)}_{\text{we want } Q-A \text{ to be sparse}} x^{(k)} + b$$

- choices :
- ① $Q = \text{diag}(A)$ (keep the sparsity)
 - ② $Q = \text{lower-triang-part}(A)$ "

Iterative methods : convergence

1st question : how can we obtain convergence of the method?

Proposition

Let $Ax = b$ be a system whose solution is $\bar{x}$. If we apply (1) from a starting point $x^{(1)}$, the error $e^{(k)} := x^{(k)} - \bar{x}$ reads as

$$e^{(k)} = (I - Q^{-1}A)e^{(k-1)}.$$

- Choice of the matrix Q :
- The best : $Q = A$ (convergence in one iteration) but requires the solution of the initial system (we are circling around...)
- Let us choose Q **close** to A

$AX=b$ choose $Q=\text{diag}(A)$ Fixed-point: $QX^{(k+1)}=(Q-A)X^{(k)}+b$

$$\text{diag}(A) = \begin{bmatrix} a_{11} & & \\ & a_{22} & \\ & & \ddots \\ & & & a_{nn} \end{bmatrix}$$

Obs: If $a_{ii}=0$ for any i , the method fails

$$Q-A = \begin{bmatrix} 0 & & -A \\ & 0 & \\ -A & & 0 \end{bmatrix}$$

$$\cancel{a_{11}} x_1^{(k+1)} = \left(-a_{12} x_2^{(k)} - a_{13} x_3^{(k)} - \dots - a_{1n} x_n^{(k)} + b_1 \right) / \cancel{a_{11}}$$

$$\cancel{a_{22}} x_2^{(k+1)} = \left(-\cancel{a_{21}} x_1^{(k)} - a_{23} x_3^{(k)} - \dots - a_{2n} x_n^{(k)} + b_2 \right) / \cancel{a_{22}}$$

$\vdots$

$$\cancel{a_{nn}} x_n^{(k+1)} = \left(-\cancel{a_{n1}} x_1^{(k)} - \cancel{a_{n2}} x_2^{(k)} - \cancel{a_{n3}} x_3^{(k)} - \dots - \cancel{a_{n,n-1}} x_{n-1}^{(k)} + b_n \right) / \cancel{a_{nn}}$$

$$Q x^{(k+1)} = \underbrace{(Q-A)}_{\text{sparsity is the same as the sparsity of } A} x^{(k)} + b$$

↓

this system
can be solved directly.

Iterative methods : the Jacobi method

- First type of system that can be solved quickly : **diagonal systems**
Leads to the **Jacobi method**.
- Choice of the diagonal matrix : $Q = D = \text{diag}(A)$
- Direct resolution of the linear system (in $\mathcal{O}(n)$)
- $x^{(k+1)} = (I - D^{-1}A)x^{(k)} + D^{-1}b$

$$x_1^{(k+1)} = \sum_{j=2}^n -\frac{a_{1j}}{a_{11}} x_j^{(k)} + \frac{b_1}{a_{11}}$$

$\vdots$

$$x_n^{(k+1)} = \sum_{j=1}^{n-1} -\frac{a_{nj}}{a_{nn}} x_j^{(k)} + \frac{b_n}{a_{nn}}$$

- Use only the values computed at the previous iteration.

$$Ax = b$$

$$A = L + D + U$$

$$\begin{array}{ccc} & \Delta & \nabla \\ & \searrow & \swarrow \end{array}$$

$$Q = L + D$$

$$Qx^{(k+1)} = (Q - A)x^{(k)} + b$$

$$Q - A = \begin{bmatrix} 0 & & & \\ & \ddots & & \\ & & -U & \\ & 0 & & 0 \end{bmatrix} = -U$$

$$a_{11} x_1^{(k+1)}$$

$$a_{21} x_1^{(k+1)} + a_{22} x_2^{(k+1)}$$

$$\vdots$$

$$a_{n1} x_1^{(k+1)} + a_{n2} x_2^{(k+1)} + \dots + a_{nn} x_n^{(k+1)}$$

$$Q = \begin{bmatrix} a_{11} & & & \\ & \ddots & & \\ & & a_{nn} & \\ a_{n1} & & & a_{nn} \end{bmatrix}$$

$$= -a_{12} x_2^{(k)} - \dots - a_{1n} x_n^{(k)} + b_1$$

$$= -a_{23} x_3^{(k)} - \dots - a_{2n} x_n^{(k)} + b_2$$

$$\vdots$$

$$b_n$$

$$\begin{array}{rcl}
 a_{11} x_1^{(k+1)} & & = -a_{12} x_2^{(k)} - \dots - a_{1n} x_n^{(k)} + b_1 \\
 a_{21} x_1^{(k+1)} + a_{22} x_2^{(k+1)} & & = -a_{23} x_3^{(k)} - \dots - a_{2n} x_n^{(k)} + b_2 \\
 \vdots & & \vdots \\
 a_{n1} x_1^{(k+1)} + a_{n2} x_2^{(k+1)} + \dots + a_{nn} x_n^{(k+1)} & = & b_n
 \end{array}$$

$$x_1^{(k+1)} = \frac{-a_{12} x_2^{(k)} - \dots - a_{1n} x_n^{(k)} + b_1}{a_{11}}$$

(Same as Jacobi)

$$x_2^{(k+1)} = \frac{-a_{21} x_1^{(k+1)} - a_{23} x_3^{(k)} - \dots - a_{2n} x_n^{(k)} + b_2}{a_{22}}$$

only diff. with Jacobi

$$x_n^{(k+1)} = \frac{(-a_{n1} x_1^{(k+1)} - \dots - a_{n,n-1} x_{n-1}^{(k+1)} + b_n)}{a_{nn}}$$

One iteration of Gauss-Seidel has the same complexity as one iteration of Jacobi

$$Q x^{(k+1)} = \underbrace{(Q-A)}_{\text{very sparse}} x^{(k)} + b$$

(same sparsity pattern as A)

Iterative methods : Gauss-Seidel

- Second type of systems that can be solved quickly : **triangular systems**
Leads to the **Gauss-Seidel method**
- If $A = L + D + U$, we take $Q = L + D$
- System to solve : $Qx^{(k+1)} = (Q - A)x^{(k)} + b$

$$\begin{pmatrix} a_{11} & 0 & 0 & \cdots & 0 \\ a_{21} & a_{22} & 0 & \cdots & 0 \\ a_{31} & a_{32} & a_{33} & \cdots & 0 \\ \vdots & \vdots & & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \\ x_3^{(k+1)} \\ \vdots \\ x_n^{(k+1)} \end{pmatrix} =$$
$$\begin{pmatrix} 0 & -a_{12} & -a_{13} & \cdots & -a_{1n} \\ 0 & 0 & -a_{23} & \cdots & -a_{2n} \\ 0 & 0 & 0 & \cdots & -a_{3n} \\ \vdots & \vdots & & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 0 \end{pmatrix} \begin{pmatrix} x_1^{(k)} \\ x_2^{(k)} \\ x_3^{(k)} \\ \vdots \\ x_n^{(k)} \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{pmatrix}$$

•

$$x_1^{(k+1)} = - \sum_{j=2}^n \frac{a_{1j}}{a_{11}} x_j^{(k)} + \frac{b_1}{a_{11}}$$

Gauss-Seidel

If we write explicitly, we finally obtain

$$x_1^{(k+1)} = - \sum_{j=2}^n \frac{a_{1j}}{a_{11}} x_j^{(k)} + \frac{b_1}{a_{11}}$$

$$x_2^{(k+1)} = - \frac{a_{21}}{a_{22}} x_1^{(k+1)} - \sum_{j=3}^n \frac{a_{2j}}{a_{22}} x_j^{(k)} + \frac{b_2}{a_{22}}$$

$\vdots$

$$x_i^{(k+1)} = - \frac{a_{i1}}{a_{ii}} x_1^{(k+1)} - \dots - \frac{a_{i,i-1}}{a_{ii}} x_{i-1}^{(k+1)} - \sum_{j=i+1}^n \frac{a_{ij}}{a_{ii}} x_j^{(k)} + \frac{b_i}{a_{ii}}$$

$\vdots$

$$x_n^{(k+1)} = - \sum_{j=1}^{n-1} \frac{a_{nj}}{a_{nn}} x_j^{(k+1)} + \frac{b_n}{a_{nn}}$$

Very similar to Jacobi.

But we use at each step the **values that have been already computed at the current iteration** !

- ① Necessary and sufficient condition for convergence
- ② Sufficient condition for convergence

① Nec. and suff. $(A\bar{x} = b)$
 Denote $\bar{x}$ as the solution
 error at iteration $k = e^{(k)} = x^{(k)} - \bar{x}$

$$\begin{aligned}
 e^{(k+1)} = x^{(k+1)} - \bar{x} &= Q^{-1}((Q-A)x^{(k)} + b) - \bar{x} \\
 &= (I - Q^{-1}A)x^{(k)} - \bar{x} + Q^{-1}b \\
 &= (I - Q^{-1}A)x^{(k)} - I\bar{x} + Q^{-1}A\bar{x} \\
 &= (I - Q^{-1}A)x^{(k)} - (I - Q^{-1}A)\bar{x} \\
 &= (I - Q^{-1}A)e^{(k)}
 \end{aligned}$$

$$e^{(k+1)} = (I - Q^{-1}A) e^{(k)}$$

$$\text{I want: } \lim_{k \rightarrow \infty} e^{(k)} = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix}$$

$$e^{(k)} \in \mathbb{R}^n$$

$$I - Q^{-1}A \in \mathbb{R}^{n \times n}$$

$$\text{If } n=1$$

$$e_{k+1} = \underbrace{(I - Q^{-1}A)}_{\in \mathbb{R}^{1 \times 1}} \cdot e_k$$

$$(|I - Q^{-1}A| < 1)$$

Thm. The iterative method converges if and only if
 all $|\lambda_i(I - Q^{-1}A)| < 1$
 Spectral radius

Assume $(I - Q^{-1}A)$ has eigenvectors $v^{(1)}, \dots, v^{(n)}$
and these eigenvectors form a basis of $\mathbb{R}^n$.

$$e^{(0)} = \alpha_1 v^{(1)} + \dots + \alpha_n v^{(n)} \quad \alpha_i \in \mathbb{R}.$$

$$\begin{aligned} e^{(1)} &= (I - Q^{-1}A) e^{(0)} = \alpha_1 (I - Q^{-1}A) v^{(1)} + \dots + \alpha_n (I - Q^{-1}A) v^{(n)} \\ &= \alpha_1 \lambda_1 v^{(1)} + \dots + \alpha_n \lambda_n v^{(n)} \end{aligned}$$

$$e^{(2)} = (I - Q^{-1}A) e^{(1)} = \alpha_1 \lambda_1^2 v^{(1)} + \dots + \alpha_n \lambda_n^2 v^{(n)}$$

$\vdots$

$$e^{(k)} = \alpha_1 \lambda_1^k v^{(1)} + \dots + \alpha_n \lambda_n^k v^{(n)}$$

'if $|\lambda_i| < 1$

$\downarrow 0$

$\downarrow 0$

$\rightarrow 0$

Acceleration of the method

In some cases, we can speed up the method by applying a well chosen **acceleration factor** ω : this is called **overrelaxation**.

$$Q = L + \omega D.$$

$\omega > 1$: help to speed up convergence

$\omega < 1$: help to make a diverging process convergent again

Equivalent to

$$x^{(k+1)} = (1 - \omega)x^{(k)} + \omega g(x^{(k)})$$

where $g(x)$ corresponds to the value computed by Gauss-Seidel at the current iteration.

If $\omega = 1$, this corresponds exactly to Gauss-Seidel.

Conditions of convergence

Proposition (Necessary and suff. condition)

If the eigenvalues λ_i of $I - Q^{-1}A$ are such that $|\lambda_i| < 1$ for all i , then the iterative method converges to the solution $\bar{x}$ of the system $Ax = b$ for every starting point $x^{(1)}$.

Proposition

Let $M = (m_{ij}) \in \mathbb{R}^{n \times n}$ be a matrix with n linearly independent eigenvectors. Then for all $i = 1, \dots, n$, there exists $t \in \{1, \dots, n\}$ such that

$$|\lambda_i| \leq \sum_{j=1}^n |m_{tj}|$$

where λ_i is an eigenvalue of M .

Every eigenvalue
is less than
the max sum of
the abs. val. of rows
of the matrix

Proposition (Sufficient condition)

We consider the system $Ax = b$. If A is a diagonally dominant matrix, namely if

$$\sum_{j \neq i} |a_{ij}| < |a_{ii}|$$

for all $i = 1, \dots, n$, then the Jacobi and Gauss-Seidel methods converge for every starting vector $x^{(1)}$.

Jacobi method:

$$Q = \begin{pmatrix} a_{11} & & \\ & \ddots & \\ & & a_{nn} \end{pmatrix}$$

$$I - Q^{-1}A$$

$$Q^{-1} = \begin{pmatrix} \frac{1}{a_{11}} & & \\ & \ddots & \\ & & \frac{1}{a_{nn}} \end{pmatrix}$$

$$I - Q^{-1}A = \begin{pmatrix} 1 & & \\ & \ddots & \\ & & 1 \end{pmatrix} - \begin{pmatrix} 1 & \frac{a_{12}}{a_{11}} & \frac{a_{13}}{a_{11}} & \dots & \frac{a_{1n}}{a_{11}} \\ \frac{a_{21}}{a_{22}} & 1 & \frac{a_{23}}{a_{22}} & \dots & \\ \vdots & & \ddots & & \vdots \end{pmatrix}$$
$$= \begin{pmatrix} 0 & -\frac{a_{12}}{a_{11}} & \dots & \\ \vdots & \ddots & & \\ & & \ddots & \\ & & & 0 \end{pmatrix}$$

$$|\lambda_i| \leq \max \left\{ \sum_{k=1, k \neq i}^n \left| -\frac{a_{ik}}{a_{ii}} \right| \right\}$$

Sufficient condition:

$$\left| -\frac{a_{12}}{a_{11}} \right| + \left| -\frac{a_{13}}{a_{11}} \right| + \dots + \left| \frac{-a_{1n}}{a_{11}} \right| < 1$$

$\vdots$

$$\left| -\frac{a_{n1}}{a_{nn}} \right| + \left| -\frac{a_{n2}}{a_{nn}} \right| + \dots + \left| \frac{-a_{n,n-1}}{a_{nn}} \right| < 1$$

$$|a_{12}| + |a_{13}| + \dots + |a_{1n}| < |a_{11}|$$

$\vdots$

The sum of all abs values of off-diag. elements
of A is less than the diag. element.
(diagon. dominant matrix)